Cryptanalysis ERASMIS - Servati et al

Review

The proposed protocol consists of different phases which are explained below:

Initialization phase In this step, the security or system administrator (SA) individually configures each object on the server, with each object having its own credential stored on the server. This scheme utilizes elliptic curve cryptography (ECC) for application in smart devices, defined as $E(F_q) = \langle p, q, a, b, n, G(P) \rangle$ and utilizing a persistent secret key $SA_{Se} \in F_q$. It is worth noting that the notations used in this paper are explained in Table 3, which defines the parameters of the elliptic curve and authentication scheme. To elaborate, ECC with parameter set $E(F_q)$ is employed to secure communications with smart devices, where each device shares a secret SA_{Se} with the security administrator (SA). The SA independently assigns credentials to all objects during the initialization phase.

Gateway and sensor node enrollment phase The following steps show how GW_j and SN_k nodes can be registered by an SA.
(1) SA opts a $GWID_j$ identity for GW_j, then calculates and saves the value $S_g = h(SA_{Se} \| GWID_j)$ in its storage. Besides, SA saves $\langle S_g, GWID_j \rangle$ in the storage of the GW_j.
(2) The system administrator provides an identity $SNID_K$ for the K th sensor node and calculates $SSN_K = h(SA_{Se} \| SNID_k)$ then keeps $\langle SNID_k, SSN_k, GWID_j \rangle$ and also stores $\langle SSN_k, GWID_j, P \rangle$ in SN_k and $\langle SNID_k, SSN_k, P \rangle$ in GW_j nodes respectively.

It should be noted that shared secret credentials are used to authenticate the gateway (GW) and sensor nodes (SN) during the users

Table 3: Main Notations

Notations.	
Symbol	Description
SA	System administrator
SA_{Se}	Secret value of the system administrator (SA)
S_g	Secret value of the gateway
PK_G	Public key of the gateway, defined as $PK_G = S_g \cdot P$
U_i	User
u_i	Random number generated by the user
SN_K	K th sensor node
GW_j	j th gateway node
SC	Smart card
$SNID_K$	Identifier of the K th sensor node (SN_K)
U_i	i th user
ID_i	Identifier of the user U_i
PW_i	Password of the user U_i
$GWID_j$	Identifier of the j th gateway node (GW_j)
SSN_k	Secret value of the K th sensor node (SN_K)
ECC	Elliptic curve cryptography
SS_A	Secret key of the system administrator (SA)
m	Number of sensors
P	Base point in elliptic curve cryptography (ECC)
n	Defined as $n = p \cdot q$
r_u, r_s	Random numbers
M	Message
F_q	A finite field consisting of q elements $\{0, 1, \ldots, q-1\}$, where q is a prime number
ΔT	Time latency
F_q^*	The set of natural numbers $\{1, \ldots, q-1\}$, i.e. $F_q^*/0$
h	Hash function
$A \stackrel{?}{=} B$	Equality check between A and B
$\oplus$	Exclusive-OR (XOR) operation
$\|\|$	Concatenation operation
Sk_u, Sk_s	Session keys on the user side and server side that must match each other

login and authentication process. These credentials allow the gateway and sensor nodes to verify each other's identities securely. Finally, the system administrator publishes the identities of registered gateway nodes so that users can access them. However, to preserve the privacy of sensor nodes, their private information is kept confidential and not disclosed. The goal is to authenticate the communication between
the gateways, sensor nodes, and users while protecting the sensitive data and identities of the sensor nodes from being exposed publicly. This approach aims to provide a secure authentication method without revealing the nodes' internal details or compromising their anonymity.

User enrollment phase Upon a successful authentication process, a trustworthy user can access the identified data. This issue occurs when the GW_j reads the sensor's observed data. The user enrollment process in the proposed protocol is accomplished as below:

1. U_i opts ID_i, and PW_i and calculates its bio hashing $b_i = H(B_i)$ with its identity ID_i and password PW_i. After that, U_i computes $HPID_i = h(ID_i \| b_i)$ and $HPW_i = h(PW_i \| b_i)$. Then it transmits $\langle HPW_i, HPID_i, GWID_j \rangle$ to SA.

2. After getting the message, the SA computes $B_1 = h(HPW_i \| HPID_i)$ then makes the $SC = \langle B_1, h(), P. \rangle$ and transmits it to the U_i through a private channel.
3. Upon receiving SC from SA, U_i selects u_i and calculates $B_1^* = h(HPW_i \| HPID_i \| u_i)$. Then U_i makes the $SC = \langle B_1, h(), P. \rangle$.

Login phase In this step, we show how the user U_i can access data via the GW_j, but first we show how the user can login to the system. The steps are as follows:

1. U_i inserts the SC into the reader or terminal and enters ID_i, u_i, and a password using biometric data B_i.
2. The smart card computes: $b_i = h(B_i), HPW_i = h(PW_i \| b_i)$, $HPID_i = h(ID_i \| HPW_i)$ and $A_1 = h(HPW_i \| HPID_i \| u_i)$.
3. The SC checks whether $A_1 \stackrel{?}{=} B_1$. When this equivalence is incorrect, the continues of scheme is halted; apart from that, SC opts a random value such a $r_u \in F_q$ and calculates: $A_2 = r_u \cdot P$ and $A_3 = r_u \cdot PK_G, A_4 = A_3 \oplus SNID_K$, and $A_5 = h(A_2 \| A_3 \| A_4 \| T_1)$ where T_1 is the current timestamp.
4. The SC transmits login messages $M_1 = \langle A_2, A_4, A_5, T_1 \rangle$ to the GW_j.

Authentication phase This stage aims to authenticate the protocol entities and facilitate generation of a shared secret key among the user (U_i), and sensor node (SN_k). The following steps detail the procedure to accomplish this:

1. $U_i \rightarrow GW_j$ When the gateway node receives a login message request, it calculates the time delay $\Delta T = T_2 - T_1$ using the gateway node's time stamp T_2, and if the time latency ΔT is not satisfactory, the login and authentication procedure fails. If the latency is reasonable, GW_j computes $A_3^* = A_2 \cdot S_g$, and computes $A_5^* = h(A_2 \| A_3^* \| A_4 \| T_1)$. Then GW_j checks whether $A_5^* \stackrel{?}{=} A_5$ is or not. If equality is correct, the protocol will be continued; otherwise, it will be halted. If so, GW_j computes $SNID_K = A_4 \oplus A_3^*$ and $A_6 = h(A_2 \| SSN_K \| GWID_j \| T_2)$. Therefore, GW_j sends $\langle A_2, A_6, T_2 \rangle$ to the SN_K.
2. $GW_j \rightarrow SN_k$ Whenever the SN_k gets a message from the GW_j, it uses its own time stamp to verify the time delay of $\Delta T = T_3 - T_2$, and the procedure is halted if this time delay is inaccurate. Otherwise, the SN_k calculates $A_6^* = h(A_2, SSN_K, GWID_j, T_2)$, and checks $A_6^* \stackrel{?}{=} A_6$, if equality is correct, the protocol will be continued; otherwise, it will be halted. SN_k opts its own random value, i.e., r_s, and calculates $A_7 = r_s \cdot .P, Sk_s = r_s \cdot A_2$, and $A_8 = h(A_6 \| A_2 \| SSN_K \| GWID_j)$. Finally, it transmits $\langle A_8, A_7 \rangle$ to the GW_j.
3. $SN_K \rightarrow GW_j$

Following receipt of M_3 from the SN_k, the GW_j confirms the time delay using its computed round trip time, and if this time latency ΔT is not reasonable, the procedure is rejected. Otherwise, it calculates $A_8^* = h(A_6 \| A_2 \| SSN_K \| GWID_j)$, and checks whether $A_8 = A_8^*$ is met or not. If it is so, the GW_j calculates $A_9 = h(A_7 \| SNID_K)$. Then GW_j transmits $\langle A_7, A_9 \rangle$ to the U_i.

4. $GW_j \rightarrow U_i$ Following the receipt of messages from the GW_j, the user examines the time delay using its round trip time and computes $A_9^* = h(A_7 \| SNID_K)$ and determines whether $A_9 \stackrel{?}{=} A_9^*$ is or not. If it is not, the session key would not be established between entities. If so, the U_i computes $Sk_u = A_7 \cdot r_u$ as its secret key with the SN_K.

The ERASMIS login and authentication phase is also shown in Fig. 2.

Update and change password phase Passwords should be modified on a regular basis to improve protection. The ERASMIS protocol takes the following steps into account for this reason:

1. U_i inserts his/her smart card into card reader and imputs his ID_i, PW_i, u_i and B_i.
2. SC computes $b_i = h(B_i), HPW_i = h(PW_i \| b_i), HPID_i = h(ID_i \| HPW_i)$ and SC also calculates $A_1 = h(HPW_i \| HPID_i \| u_i)$ and checks whether $B_1 \stackrel{?}{=} A_1$. If it is not so, the SC halts the session; otherwise, the SC lets U_i to enter a new password. U_i 's new password, which he or she chooses and enters into the smart card, is represented by PW_i^{new} .
3. The smart card calculates $HPW_i^{\text{new}} = h(PW_i^{\text{new}} \| b_i), HPID_i^{\text{new}} = h(ID_i \| HPW_i^{\text{new}})$, and $B_1^{\text{new}} = h(HPW_i^{\text{new}} \| HPID_i^{\text{new}} \| u_i)$.

Finally, SC replaces B_1, with B_1^{new}. In addition, the described procedure in Algorithm 1 illustrates how the password is updated in the ERASMIS's protocol.

Sensor node addition In situations where the SN_K is cracked or hacked by an intruder, or SN_K loses its ability to store energy, it should be replaced. The steps for adding a new sensor node are explained below.

1. SA opts a new identity $SNID_K^{\text{new}}$ for the SN_K.
2. After that, SA computes a secret value i.e. $SSN_K = h(SA_{Se} \| SNID_K^{new})$.
3. Finally, SA saves $\langle SNID_K, SSN_K \rangle$ into memory of the GW_j and $\langle SSN_K \rangle$ in to memory of SN_K.

Algorithm 1 Algorithm of updating the password in ERASMIS protocol.
Data: Personality information such as $\langle ID_i, PW_i, B_i \rangle$
Result: Update password

1. U_i inserts his SC and puts ID_i, PW_i, u_i and his/her bio-metric B_i.
2. SC calculates $b_i = h(B_i), HPW_i = h(PW_i \| b_i)$ and $HPID_i = h(ID_i \| HPW_i)$.
3. SC also calculates $A_1 = h(HPW_i \| HPID_i \| u_i)$.
4. if $(A_1 == B_1)$ then
   (a) U_i enters his new password PW_i^{new} and u_i into SC.

(b) SC calculates $HPW_i^{\text{new}} = h\left(PW_i^{\text{new}} \| b_i\right), HPID_i^{\text{new}} = h\left(ID_i \| HPW_i^{\text{new}}\right), B_1^{\text{new}} = h\left(HPW_i^{\text{new}} \| HPID_i^{\text{new}} \| u_i\right)$.

(c) SC substitutes old B_1 with B_1^{new} .

5. else
6. SC terminates the session.
7. end if

Cryptanalysis of Servati et al Scheme

1. Biometric Hashing

The protocol utilizes $b_i = h(B_i)$ where B_i is biometric data.

Analysis: Cryptographic hash functions suffer from the avalanche effect. A single bit change in input results in a completely different output. Biometric scans are noisy and never yield bit-identical results twice.

Example: If the same biometric is used across different services, the hashed value b_i becomes a global identifier, violating user privacy.

Impact: Legitimate users will permanently fail authentication because $h(B_i^{login}) \neq h(B_i^{enroll})$. The protocol requires a Fuzzy Extractor or Biometric Locking mechanism, which is absent.

2. The "Ghost User" Authentication Failure (Impersonation Attack) (Absence of Source Authentication)

Classification: Security Flaw / Critical Vulnerability Target: Mutual Authentication

The Vulnerability: The most severe flaw in ERASMIS is that the Gateway (GW_j) never verifies the identity of the User (U_i). In the Login Phase, the User sends $M_1 = \langle A_2, A_4, A_5, T_1 \rangle$. $A_2 = r_u \cdot P$ (Random ECC point). $A_4 = (r_u \cdot PK_G) \oplus SNID_k$ (Masked Sensor ID). $A_5 = h(A_2 \parallel (r_u \cdot PK_G) \parallel A_4 \parallel T_1)$ (Integrity Hash).

The Attack (Scientific Walkthrough):
1. Adversary Setup: An attacker (Adv) captures a valid Sensor ID ($SNID_k$) via eavesdropping or physical inspection of the medical device (IoT IDs are rarely secret). The Adv also knows the Gateway's public key (PK_G) and system params.
2. Attack Execution: Adv generates a random nonce r_{adv}. Adv computes $A_2' = r_{adv} \cdot P$. Adv computes the shared secret $K_{adv} = r_{adv} \cdot PK_G$. Adv computes $A_4' = K_{adv} \oplus SNID_k$. Adv computes $A_5' = h(A_2' \parallel K_{adv} \parallel A_4' \parallel T_{current})$. Adv sends $\langle A_2', A_4', A_5', T_{current} \rangle$ to the Gateway.
3. Gateway Processing: The GW computes $K^* = A_2' \cdot S_g = r_{adv} \cdot P \cdot S_g = K_{adv}$. The GW verifies A_5'. It matches. The GW extracts $SNID_k = A_4' \oplus K^*$. It is a valid ID.

4. Result: The Gateway accepts the request.
Conclusion: The protocol checks if the sender knows a Sensor ID. It does not check if the sender is a registered User (U_i). The hash A_5 contains no user-specific secrets (ID_i, PW_i, b_i). Thus, ERASMIS fails to provide source authentication.

3. The "Stateless Verification" Impossibility

Type: Critical Logic Error Location: Authentication Phase, Step 3 ($SN_K \rightarrow GW_j$)

Analysis: In Step 3, the Sensor Node sends $M_3 = \langle A_8, A_7 \rangle$ to the Gateway. The Gateway must verify:

$$A_8 \stackrel{?}{=} h(A_6 \parallel A_2 \parallel SSN_K \parallel GWID_j)$$

To perform this verification, the Gateway requires the nonce A_2 (generated by the user in Step 1). However: 1. The Gateway is typically modeled as stateless to handle high IoT throughput. 2. The protocol does not describe any state storage mechanism binding the incoming M_3 to the initial M_1. 3. Crucially, M_3 does not echo A_2 back to the Gateway. Consequence: When the Gateway receives M_3, it mathematically cannot compute the hash A_8 because it does not possess A_2. The verification step is impossible.

4. Gateway Man-in-the-Middle (Lack of End-to-End Security)

Type: Trust Model Violation / Architecture Flaw Target: Session Key Integrity

Analysis: The protocol claims to establish a session key Sk_u between User (U_i) and Sensor (SN_K). However, the User verifies the Sensor's key contribution (A_7) using the parameter A_9, which is generated by the Gateway, not the Sensor:

$$A_9 = h(A_7 \parallel SNID_K)$$

Attack Vector: A compromised Gateway can intercept the Sensor's real A_7 ($r_s \cdot P$) and replace it with $A_7' = r_{evil} \cdot P$. The Gateway then generates a valid A_9' matching the forged A_7'. Consequence: The User establishes a key with the Gateway (thinking it is the Sensor). The Gateway can decrypt all patient data before re-encrypting it for the Sensor. There is no End-to-End security.

5. Destination Confusion (Binding Attack)

Classification: Logic Flaw Target: Data Routing

The Flaw: The User sends a request intending to reach Sensor A ($SNID_A$). A malicious Gateway can route this to Sensor B ($SNID_B$) and fool the User into establishing a connection with B, believing it is A.

Scientific Analysis: 1. User sends A_4 (masked $SNID_A$). 2. Malicious Gateway decrypts $SNID_A$ but decides to connect to $SNID_B$. 3. Gateway sends challenge to SN_B. SN_B replies with A_7^B. 4. Gateway computes $A_9 = h(A_7^B \parallel SNID_A)$. Note: GW puts $SNID_A$ in the hash, even though A_7^B came from B. 5. User verifies A_9. It passes. 6. User sends data encrypted with the shared key. SN_B receives it. 7. Impact: In a medical setting, this is catastrophic. A "Stop Pace" command intended for Patient A could be successfully routed to and executed by Patient B's device. The protocol fails to cryptographically bind the Sensor's identity to the Sensor's key contribution (A_7) in a way the Gateway cannot forge.

6. Offline Password Guessing (Smart Card Loss)

Type: Credential Theft

Analysis: If an attacker steals the Smart Card (SC) and extracts data via side-channel attacks (a standard threat model assumption): Extracted: $B_1 = h(h(PW_i \parallel h(B_i)) \parallel h(ID_i \parallel \dots) \parallel u_i)$. Because ID_i is usually low entropy (e.g., patient ID) and PW_i is low entropy, the only strong factor is B_i. However, due to the flaw in 2.3 (lack of fuzzy extraction), if the attacker obtains a latent fingerprint (common on the stolen card itself), they can brute force the hash chain offline to verify guesses.

7. Missing Entity Authentication in Key Establishment

Flaw: The protocol doesn't provide explicit key confirmation. While A_9 serves as implicit authentication, there's no proper key confirmation handshake.

Example: An active attacker could modify A_7 in transit, causing user and sensor to compute different session keys without detection until actual data transmission fails.

8. Denial of Service (DoS) and Operational Deficiencies

- **Anonymous Response DoS (Computational Amplification)** Type: DoS Vulnerability Location: Authentication Phase, Step 3
  Analysis: The Sensor's response $M_3 = \langle A_8, A_7 \rangle$ contains no identifier ($SNID_K$). Consequence: Upon receiving M_3, the Gateway does not know which sensor sent it. To verify A_8 (which involves SSN_K), the Gateway must iterate through all m registered sensors, performing m hash operations for every incoming packet. An attacker can flood the Gateway with fake M_3 packets, causing CPU exhaustion.

- **Battery Exhaustion via Replay** Type: Resource Depletion Target: Sensor Nodes
  Analysis: The protocol relies on timestamps (T_2, T_3) but does not specify strict stateful replay protection (e.g., storing the last timestamp). Attack Vector: An attacker captures a valid $GW \rightarrow SN$ command and replays it multiple times within the valid time window ΔT. Consequence: The Sensor performs expensive ECC scalar multiplications ($r_s \cdot P$) for each replay, rapidly

draining the battery of the medical device.

9. Revocation Impossibility

Type: Operational Deficiency Target: Lifecycle Management

Analysis: The authentication logic relies on shared secrets embedded in the Gateway and Sensor. There is no check against a dynamic User Revocation List during the Login Phase (which only checks A_5). Consequence: Revoking a lost Smart Card or a compromised user is impossible without changing the global keys of the Gateway or re-keying every sensor node in the system.

10. Gateway as Single Point of Failure

Type: Architectural Weakness Target: System Resilience

Analysis: The Gateway stores the master secrets (SSN_k) for all sensors. Consequence: If the Gateway is physically compromised, the attacker obtains the master keys for the entire medical network, breaking the security of all past and future communications for all patients.

11. Sensor Anonymity Contradiction via Ephemeral Leakage

Type: Privacy Breach / Side-Channel Vulnerability Target: Sensor Location Privacy

Scientific Analysis: The protocol claims to protect the identity of the sensor ($SNID_K$). However, in the Login Phase, the User transmits:

$$A_3 = r_u \cdot PK_G$$

$$A_4 = A_3 \oplus SNID_K$$

The Flaw: If the User's ephemeral random number generator is weak, or if r_u is leaked via a side-channel attack on the User's device (a standard IoT threat model), an attacker can compute:

$$SNID_K = A_4 \oplus (r_u \cdot PK_G)$$

Conclusion: The confidentiality of the Sensor ID is not cryptographically robust; it relies entirely on the secrecy of the ephemeral nonce r_u, violating the principle that long-term privacy should not depend on ephemeral state.

12. Operational Failure: Strict Clock Synchronization Dependency

Type: Availability / Robustness Defect Target: IoT Network Stability

Scientific Analysis: The protocol relies heavily on timestamp checks ($\Delta T = T_{current} - T_{received}$). The Deficiency: Medical Wireless Sensor Networks (MWSNs) often operate in environments with variable latency or on hardware with cheap crystal oscillators that drift significantly. Conclusion: The protocol lacks a "Resynchronization Mechanism." If a legitimate sensor's clock drifts beyond ΔT, it will be permanently locked out of the network (DoS by design) until manually serviced. This is a critical operational defect for medical systems.

13. Lack of Explicit Key Confirmation

Type: Protocol Design Deficiency Target: Session Reliability

Scientific Analysis: The protocol establishes a session key Sk_u, but the User never sends a confirmation message to the Sensor proving they have derived the correct key. The Flaw: If an active attacker modifies the key material (A_7) in transit, the User and Sensor will derive different keys ($Sk_u \neq Sk_s$). Conclusion: The failure will only be detected when the first medical data packet is sent and fails decryption. A proper protocol must include a "Key Confirmation Hash" (e.g., $h(Sk_u \parallel "Confirmation")$) before sensitive data is transmitted to prevent data corruption.

14. Insufficient Key Agreement and Forward Secrecy

Flaw: The protocol claims perfect forward secrecy but the session key derivation appears vulnerable. The session key $Sk = r_s \cdot r_u \cdot P$ is computed from ephemeral secrets, but the authentication messages don't adequately protect against key compromise impersonation attacks.

Example Attack: An adversary who compromises r_u (perhaps through side-channel attacks on the smart card) can compute $A_3 = r_u \cdot PK_G$ and thus recover $SNID_K = A_4 \oplus A_3$. This breaks sensor anonymity and allows impersonation.

15. Lack of Perfect Forward Secrecy (PFS) regarding Gateway Compromise

Type: Security Weakness

Analysis: If the Gateway's long-term secret key S_g is compromised: 1. The attacker can compute $A_3 = A_2 \cdot S_g =$

$r_u \cdot P \cdot S_g$ for all past recorded sessions. 2. However, the Session Key (Sk_u) is defined as $A_7 \cdot r_u = r_s \cdot P \cdot r_u$. 3. To solve this, the attacker needs r_u or r_s. 4. Since $A_2 = r_u P$ and $A_7 = r_s P$ are public, finding r_u is the Discrete Logarithm Problem (DLP).

Verdict: The protocol actually does provide PFS for the session key, which is a strength. However, if S_g is lost, the attacker can decrypt the identity of the target sensor ($SNID_k$) from all past login messages (A_4), violating anonymity.

16. Known-Key Secrecy Violation (KCI)

Type: Cryptographic Weakness Target: Forward Secrecy

Scientific Analysis: The session key is calculated as $Sk = r_u \cdot r_s \cdot P$. The Flaw: This construction does not mix long-term keys into the session key derivation function (KDF). It relies only on ephemeral values. Conclusion: While this provides Perfect Forward Secrecy (PFS), it fails to provide resilience against Key Compromise Impersonation (KCI). If the User's long-term key is compromised, an attacker can trivially spoof sessions, as the session key doesn't mathematically bind the long-term identity to the ephemeral session.

17. Lack of Explicit Mutual Authentication

Flaw: While the protocol claims mutual authentication, the authentication flows are asymmetric and don't clearly establish mutual proof of possession of long-term keys.

Example: The gateway authenticates the user, but the user's authentication of the gateway is weak—relying primarily on the correct computation of A_9.

18. Timestamp Dependency and Replay Protection Weakness

Flaw: Heavy reliance on synchronized clocks and simple timestamp checks (ΔT) is problematic for distributed IoT systems.

Example: In poorly synchronized networks or under delay attacks, legitimate messages might be rejected while carefully timed replays could succeed within the ΔT window.

19. The "Missing Parameter" Availability Paradox

Type: Functional Defect / Unimplementable Algorithm Location: Login Phase, Step 3

Analysis: The protocol requires the User's Smart Card (SC) to compute the parameter A_4 during the Login Phase:

$$A_4 = A_3 \oplus SNID_k$$

To perform this XOR operation, the User must possess the Sensor Node Identity ($SNID_k$). However, a review of the User Enrollment Phase (Section 4.3) reveals that the Smart Card is initialized only with $\langle B_1, h(\cdot), P \rangle$. The $SNID_k$ is neither stored on the card nor provided by the user inputs (ID_i, PW_i, B_i). Consequence: A legitimate user has absolutely no way to obtain the target $SNID_k$ to compute A_4. The protocol cannot be executed as written.

20. The "Missing Parameter" Paradox

Type: Functional Defect / Unimplementable Algorithm

Analysis: In the Login Phase, the User computes $A_4 = A_3 \oplus SNID_k$. This calculation requires the User to possess the $SNID_k$ (Sensor Node Identity).

However, looking at the User Enrollment Phase (Section 4.3) and Initialization Phase, the $SNID_k$ is never provided to the User or stored in the Smart Card (SC). The SC contains: $\langle B_1, h(\cdot), P \rangle$. The User inputs: ID_i, PW_i, B_i.

Conclusion: The User has absolutely no way to know which $SNID_k$ they are communicating with or to perform the XOR operation to generate A_4. The protocol cannot be executed as written.

21. The "Impossible Execution" Paradox (Missing Parameter)

Classification: Functional Defect / Logic Error Target: Protocol Completeness

The Defect: In Section 4.4 (Login Phase), Step 3, the protocol requires the Smart Card (SC) to compute:

$$A_4 = A_3 \oplus SNID_k$$

This requires the User or the SC to possess the Sensor Node Identity ($SNID_k$).

Rigorous Analysis of Preconditions: Initialization: $SNID_k$ is stored in GW_j and SN_k. User Enrollment (Section 4.3): The User sends $HPID_i$ and HPW_i. The SA returns a Smart Card containing $\langle B_1, h(\cdot), P \rangle$. The Paradox: At no point is $SNID_k$ provisioned to the User or the Smart Card. Result: When a real user attempts to use this system, they cannot compute A_4 because $SNID_k$ is an undefined variable in the User's memory space. The protocol cannot be executed.

22. Functional Defect: The Missing Parameter ($SNID_k$)

In Section 4.4 (Login Phase), Step 3, the Smart Card computes:

$$A_4 = A_3 \oplus SNID_k$$

However, according to Section 4.3 (User Enrollment), the Smart Card is initialized with:

$$SC = \langle B_1, h(\cdot), P \rangle$$

Defect: The Smart Card does not store $SNID_k$. The User does not input $SNID_k$. Therefore, the User cannot compute A_4. The protocol is logically broken and cannot be implemented as described.

Insider Attack Vulnerability Since the User Authentication relies entirely on verifying knowledge of $SNID_k$ (via the XOR in A_4), any privileged insider (e.g., a former employee) who knows the $SNID_k$ values can impersonate any user or simply access the sensors directly through the Gateway without valid user credentials.

23. Gateway Compromise leads to Total System Collapse

Classification: Architecture Flaw / Security Shortcoming Target: Resilience

The Vulnerability: In the Enrollment Phase (Section 4.2), the paper states: > "SA... stores $\langle SNID_k, SSN_k, GWID_j \rangle$ ... in GW_j."

Here, SSN_k is the master secret of the Sensor Node. Analysis: The Gateway is an aggregator node, often placed in semi-trusted environments (hospital hallways). If the Gateway is physically compromised or hacked: 1. The attacker extracts the database. 2. The attacker obtains SSN_k for every sensor in the network. 3. Result: The attacker can impersonate any sensor, decrypt all past/future communications, and inject false medical data. The Gateway should only store a verifier (hash) or public key, not the master secret of the nodes it manages.

24. Lack of Source Authentication (Impersonation)

The protocol does not cryptographically bind the Login Request (M_1) to the User's identity (ID_i) or credentials (PW_i). Analysis: The Gateway validates the request by checking A_5. However, A_5 is generated using only random nonces and the Sensor ID ($SNID_k$). Attack Vector: An adversary who knows a target Sensor ID ($SNID_k$) and the Gateway's public key can generate a valid M_1 packet. The Gateway will accept this packet, believing it comes from a valid user. Consequence: Complete breach of access control. Unauthorized entities can establish sessions with medical sensors.

Missing Execution Parameter ($SNID_k$) The protocol requires the User to compute $A_4 = A_3 \oplus SNID_k$ in the Login Phase. Deficiency: The User Enrollment phase does not provision $SNID_k$ to the User's Smart Card. The User inputs are only ID, PW, Bio. Consequence: A legitimate user has no way to obtain $SNID_k$. The XOR operation is impossible to perform, and the protocol cannot be executed.

25. The "Stateless Verification" Impossibility (Protocol Correctness Failure)

Classification: Critical Functional Defect / Logic Error Location: Authentication Phase, Step 3 ($SN_K \rightarrow GW_j$)

The Flaw: In Step 3, the Sensor Node (SN_K) sends the message $M_3 = \langle A_8, A_7 \rangle$ to the Gateway. The Gateway is required to verify:

$$A_8 \stackrel{?}{=} h(A_6 \parallel A_2 \parallel SSN_K \parallel GWID_j)$$

Scientific Analysis: To perform this verification, the Gateway requires access to the variables A_6 and A_2. 1. A_2 and A_6 were generated/received in Step 1 (Login Phase). 2. The protocol does not specify that the Gateway maintains a session state. In standard IoT protocols (like UDP/CoAP), Gateways are often stateless to handle high throughput. 3. Crucially, the message M_3 does not echo A_2 back to the Gateway. 4. The Failure: When the Gateway receives $\langle A_8, A_7 \rangle$, it has no context. It does not possess A_2 or A_6 from the current packet. Unless the Gateway commits significant memory resources to store the state of every pending login request (indexed by what?), it mathematically cannot compute the hash to verify A_8.

Impact: The protocol is broken. Without state maintenance (which introduces DoS risks) or echoing parameters, the verification step is impossible.

26. The "Anonymous Response" Identification Failure (Computational DoS)

Classification: Efficiency Defect / Denial of Service Vulnerability Location: Authentication Phase, Step 3

The Flaw: The response message from the Sensor is $M_3 = \langle A_8, A_7 \rangle$. Note that this message contains no identifier ($SNID_K$) for the Sensor Node.

Scientific Analysis: 1. The Gateway manages m sensors. 2. Upon receiving $\langle A_8, A_7 \rangle$, the Gateway must verify A_8. 3. The calculation of A_8 involves SSN_K (the sensor's secret key). 4. Since the Gateway does not know which sensor sent the message (no ID is included), the Gateway must iterate through all m registered sensors. 5. It must compute $h(A_6 \parallel A_2 \parallel SSN_i \parallel \dots)$ for $i = 1$ to m until a match is found.

Attack Scenario (Computational DoS): An attacker floods the Gateway with fake $\langle A_8, A_7 \rangle$ packets. For each packet,

the Gateway is forced to perform m hash operations (where m could be thousands in a hospital). This creates a massive computational amplification vector, trivially crashing the Gateway.

.

27. Lack of End-to-End Security (Gateway MitM)

Classification: Trust Model Violation / Security Weakness
Target: Session Key Integrity

The Flaw:
The protocol claims to establish a session key between User (U_i) and Sensor (SN_K). However, the cryptographic binding is entirely dependent on the Gateway's honesty.

Scientific Analysis:
1. The User computes $Sk_u = r_u \cdot A_7$. The User receives A_7 from the Gateway.
2. The User verifies $A_9 = h(A_7 \parallel SNID_K)$. This hash is generated by the Gateway, not the Sensor.
3. The Attack: A compromised Gateway can intercept the Sensor's real A_7 (r_sP) and replace it with $A'_7 = r_{evil}P$.
4. The Gateway generates a valid $A'_9 = h(A'_7 \parallel SNID_K)$ and sends it to the User.
5. The User accepts the key $Sk' = r_u \cdot r_{evil} \cdot P$.
6. The Gateway creates a separate key with the Sensor.

Conclusion: The User has zero cryptographic proof that the key negotiation parameter A_7 actually originated from the Sensor. The protocol relies on "Hop-by-Hop" security rather than "End-to-End" security. In a threat model where the Gateway (a general-purpose router) might be compromised, the medical data encryption is broken.

28. Sensor De-synchronization via Replay

Classification: Availability Attack Location: Sensor Node State

The Flaw:
The protocol uses timestamps (T_2, T_3) but does not specify that the Sensor must store the last seen timestamp to prevent replays strictly.

Scientific Analysis:
1. If the Sensor accepts any message within a time window ΔT (e.g., 5 seconds to account for network jitter), an attacker can capture a valid Login command ($GW \rightarrow SN$) and replay it multiple times within that 5-second window.
2. The Sensor will perform the computationally expensive scalar multiplication (r_sP) and key generation for each replay.
3. This drains the battery of the resource-constrained medical sensor (Battery Exhaustion Attack).